Yelp Review Analysis: From Baselines to Business Intelligence Agent

A four-stage NLP pipeline progressing from feature engineering to agentic RAG systems

Github: github.com/Yuhanthegod/Yelp Project

Project Overview

This project applies a progressive four-stage NLP pipeline to the Yelp Academic Dataset (50,000 reviews). Each stage builds on the previous, exploring feature engineering and capability evolve as methods advance from traditional feature engineering to a fully agentic retrieval augmented system.

Stage 1	Stage 2	Stage 3	Stage 4
Baseline Comparison	**Deep Learning**	**LLM Evaluation**	**BI Agent**
TF-IDF + Gaussian Naive Bayes, Sentence-BERT + logistic regression	1D-CNN, Bidirectional LSTM, Transformer Encoder, Fine-tuned DistilBERT	3-shot classification, Chain-of-thought reasoning with Qwen 8B	RAG + tool-augmented ReAct agent with three specialized retrievers

Key progression: feature engineering → Freeze pretrained representations → End-to-end transformer fine-tuning → Direct LLM prompting → Agentic retrieval augmented systems

Stage 1 — Baseline Comparison

Three modeling paradigms are compared on the 5-class star rating prediction task (test set).

Method	Accuracy	Macro F1
TF-IDF + Logistic Regression	56.92%	56.10%
Sentence-BERT + Logistic Regression	53.25%	52.09%
BERT (Fine-tuned)	**62.58%**	**62.16%**

Why does fine-tuned BERT win? End-to-end training jointly optimizes the text representation and the classification boundary for this specific task. Frozen sentence embeddings must compromise between both objectives at once, so any improvement the pretraining objective and star-rating classification cannot be combined.

Why does Sentence-BERT underperform TF-IDF? Sentence-BERT embeddings are optimized for semantic similarity between sentence pairs, not for single-sentence classification. Star ratings are driven by specific sentiment vocabulary ("terrible", "amazing", "overpriced") — patterns that TF-IDF's word-frequency statistics capture directly. Sentence-BERT compresses them into a dense vector designed for different purpose.

Stage 2 — BERT Fine-Tuning Experiments

A systematic hyperparameter search across **216 configurations** covers all combinations of the following dimensions:

Dimension	Values
Backbone model	distilroberta-base, uncased , bert-base-uncased , roberta-base
Seq. length	Binary (2 class): 256, 512
Pooling	Classification (cross-entropy) Regression (MSE)
Learning rate	1e-5, 2e-5
Each size	4, 8, 16
Warmup ratio	0, 0.1
Total combinations	**3 × 3 × 2 × 2 × 2 × 2 × 2 × 2 = 216**

Each sun saves validation and test metrics, classification reports, confusion matrices, and predictions files. Our top configurations, training curves, error analysis, model checkpoints are not saved during Stage 2 — only aggregate outputs are retained. The best 5-class configuration is re-trained and saved in Stage 4 (Step 0).

BERT (Fine-tuned)

Metric	Test Accuracy	Test Macro F1	MAE
DistilBERT base uncased	56.04%	56.55%	—
BERT base uncased	57.58%	57.90%	—
RoBERTa base	**57.82%**	**57.91%**	**0.5968**

5 class → test set

Method	Test Accuracy	Test Macro F1	Off-by-1 Acc.
DistilBERT base uncased	65.49%	65.36%	97.25%
BERT base uncased	66.75%	66.30%	97.49%
RoBERTa-base	**66.56%**	**66.49%**	**98.10%**

Off-by-1 Accuracy evaluates for star-rating prediction, a prediction is counted as "off-by-1 correct" if it falls within ±1 star of the true rating. Predicting a 4 when the true label is 5 or 3 is 1 off. This metric captures near-miss predictions that are practically acceptable — a user who rates a restaurant 4 and a model that predicts 3 or 5 is still producing a useful recommendation, unlike predictions that are off by 2 or more. It provides a more forgiving measure of ordinal ability of distinguishing adjacent star ratings, where the linguistic signals in the reviews are often ambiguous.

Why RoBERTa over BERT and DistilBERT? RoBERTa removes the Next Sentence Prediction task from pre-training, uses dynamic masking, and trains on significantly more data. This produces richer token representations that transfer more effectively to downstream tasks. DistilBERT, while faster, loses expressive capacity through knowledge distillation.

Why 5 class vs binary? Binary classification collapses the ordinal structure entirely — a 1-star review and a 4-star review both collapse into "positive" and "negative", discarding the intermediate signal. 5-class preserves ordinal structure, allowing the model to learn that 1 vs 2 is a harder distinction than 1 vs 5. Binary is useful for sentiment analysis, but loses information that matters for recommendation ranking.

Why 5 class vs text-level? Text-level prediction asks the model to predict the rating of a single review from its text alone. This is harder because reviewers with different baselines express the same sentiment differently — some reviewers never give 5 stars, others give 5 stars for mediocre experiences. Business-level aggregation provides context that normalizes for these baseline differences.

Why RoBERTa over BERT and DistilBERT? RoBERTa removes the Next Sentence Prediction task from pre-training, uses dynamic masking, and trains on significantly more data. This produces richer token representations that transfer more effectively to downstream tasks. DistilBERT, while faster, loses expressive capacity through knowledge distillation.

Why 5 class vs binary? Binary classification collapses the ordinal structure entirely — a 1-star review and a 4-star review both collapse into "positive" and "negative", discarding the intermediate signal. 5-class preserves ordinal structure, allowing the model to learn that 1 vs 2 is a harder distinction than 1 vs 5. Binary is useful for sentiment analysis, but loses information that matters for recommendation ranking.

Why 5 class vs text-level? Text-level prediction asks the model to predict the rating of a single review from its text alone. This is harder because reviewers with different baselines express the same sentiment differently — some reviewers never give 5 stars, others give 5 stars for mediocre experiences. Business-level aggregation provides context that normalizes for these baseline differences.

Why 5 class vs text-level? Text-level prediction asks the model to predict the rating of a single review from its text alone. This is harder because reviewers with different baselines express the same sentiment differently — some reviewers never give 5 stars, other give 5 stars for mediocre experiences. Business-level aggregation provides context that normalizes for these baseline differences.

Why RoBERTa over BERT and DistilBERT? RoBERTa removes the Next Sentence Prediction task from pre-training, uses dynamic masking, and trains on significantly more data. This produces richer token representations that transfer more effectively to downstream tasks. DistilBERT, while faster, loses expressive capacity through knowledge distillation.

Why 5 class vs binary? Binary classification collapses the ordinal structure entirely — a 1-star review and a 4-star review both collapse into "positive" and "negative", discarding the intermediate signal. 5-class preserves ordinal structure, allowing the model to learn that 1 vs 2 is a harder distinction than 1 vs 5. Binary is useful for sentiment analysis, but loses information that matters for recommendation ranking.

Why 5 class vs text-level? Text-level prediction asks the model to predict the rating of a single review from its text alone. This is harder because reviewers with different baselines express the same sentiment differently — some reviewers never give 5 stars, other give 5 stars for mediocre experiences. Business-level aggregation provides context that normalizes for these baseline differences.

Why RoBERTa over BERT and DistilBERT? RoBERTa removes the Next Sentence Prediction task from pre-training, uses dynamic masking, and trains on significantly more data. This produces richer token representations that transfer more effectively to downstream tasks. DistilBERT, while faster, loses expressive capacity through knowledge distillation.

Why 5 class vs binary? Binary classification collapses the ordinal structure entirely — a 1-star review and a 4-star review both collapse into "positive" and "negative", discarding the intermediate signal. 5-class preserves ordinal structure, allowing the model to learn that 1 vs 2 is a harder distinction than 1 vs 5. Binary is useful for sentiment analysis, but loses information that matters for recommendation ranking.

Why 5 class vs text-level? Text-level prediction asks the model to predict the rating of a single review from its text alone. This is harder because reviewers with different baselines express the same sentiment differently — some reviewers never give 5 stars, other give 5 stars for mediocre experiences. Business-level aggregation provides context that normalizes for these baseline differences.

Why RoBERTa over BERT and DistilBERT? RoBERTa removes the Next Sentence Prediction task from pre-training, uses dynamic masking, and trains on significantly more data. This produces richer token representations that transfer more effectively to downstream tasks. DistilBERT, while faster, loses expressive capacity through knowledge distillation.

Why 5 class vs binary? Binary classification collapses the ordinal structure entirely — a 1-star review and a 4-star review both collapse into "positive" and "negative", discarding the intermediate signal. 5-class preserves ordinal structure, allowing the model to learn that 1 vs 2 is a harder distinction than 1 vs 5. Binary is useful for sentiment analysis, but loses information that matters for recommendation ranking.

Why 5 class vs text-level? Text-level prediction asks the model to predict the rating of a single review from its text alone. This is harder because reviewers with different baselines express the same sentiment differently — some reviewers never give 5 stars, other give 5 stars for mediocre experiences. Business-level aggregation provides context that normalizes for these baseline differences.

Why RoBERTa over BERT and DistilBERT? RoBERTa removes the Next Sentence Prediction task from pre-training, uses dynamic masking, and trains on significantly more data. This produces richer token representations that transfer more effectively to downstream tasks. DistilBERT, while faster, loses expressive capacity through knowledge distillation.

Why 5 class vs binary? Binary classification collapses the ordinal structure entirely — a 1-star review and a 4-star review both collapse into "positive" and "negative", discarding the intermediate signal. 5-class preserves ordinal structure, allowing the model to learn that 1 vs 2 is a harder distinction than 1 vs 5. Binary is useful for sentiment analysis, but loses information that matters for recommendation ranking.

Why 5 class vs text-level? Text-level prediction asks the model to predict the rating of a single review from its text alone. This is harder because reviewers with different baselines express the same sentiment differently — some reviewers never give 5 stars, other give 5 stars for mediocre experiences. Business-level aggregation provides context that normalizes for these baseline differences.

Why RoBERTa over BERT and DistilBERT? RoBERTa removes the Next Sentence Prediction task from pre-training, uses dynamic masking, and trains on significantly more data. This produces richer token representations that transfer more effectively to downstream tasks. DistilBERT, while faster, loses expressive capacity through knowledge distillation.

Why 5 class vs binary? Binary classification collapses the ordinal structure entirely — a 1-star review and a 4-star review both collapse into "positive" and "negative", discarding the intermediate signal. 5-class preserves ordinal structure, allowing the model to learn that 1 vs 2 is a harder distinction than 1 vs 5. Binary is useful for sentiment analysis, but loses information that matters for recommendation ranking.

Why 5 class vs text-level? Text-level prediction asks the model to predict the rating of a single review from its text alone. This is harder because reviewers with different baselines express the same sentiment differently — some reviewers never give 5 stars, other give 5 stars for mediocre experiences. Business-level aggregation provides context that normalizes for these baseline differences.

Best configuration (1-class): RoBERTa-base · Lr=1e-5 · batch=4 · epochs=4. This checkpoint is reused as the reviewer backbone for Stage 3.

Stage 3 — LLM Evaluation and Fine-Tuning

Two instruction-tuned LLMs are evaluated on the same rating task through a three-tier progression.

Qwen3-2.7B Instruct

- Tier 1: Zero-shot direct
- Tier 3: LoRA fine-tune (QLoRA) (16%)

Qwen3-8B (Ollama)

- Tier 1: Zero-shot direct
- Tier 2: Few-shot chain-of-thought prompting
- Tier 3: LoRA fine-tune + thinking chains

Fine-Tuning Details

Qwen3-2.7B-Instruct — two hardware configurations:

Parameter	20GB Universe (RTX 3080)	MPS (M1Pro 32GB)
Quantization	4-bit NF4 quant (BitsandBytes compute)	None — full bf16
LoRA rank r	16	32
LoRA alpha	32	64
LoRA dropout	0.05	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj (7 modules)	
Learning rate	2e-4	2e-4
Epochs	2	2
Each size effective	4 × 4 = 16	8 × 8 = 32
Gradient checkpoint	Cosmic warmup rate 5%	—
Max seq len (train)	256 tokens (reference: 1024)	—
Reported results	—	2 higher throughput

Both configurations apply LoRA adapters to all 7 projection layers (attention + FFN) gate/up/down). Sequence packing is available during training to eliminate padding waste. The model is trained with thinking chains, reference uses a separate 1024 token context window to avoid truncating long Yelp reviews.

Why Qwen3-8B (Ollama)? Ollama provides a local inference server with OpenAI-compatible API, enabling rapid prompt iteration without GPU memory management. Qwen3-8B offers stronger reasoning than the 2.7B variant while remaining fast enough for interactive development. The thinking mode produces explicit reasoning traces that can be analyzed for prompt design.

Fine-tuning is reasoning model requires preserving its chain-of-thought structure. A dedicated two-stage pipeline is used:

Step	Purpose
1 — Generate thinking traces	Run DeepSeek on the training set, extract <think>...</think> traces from correctly predicted examples
2 — LoRA fine-tuning	Train on thinking chain data with cosmic hyperparameters (reserve reasoning ability

Training format for the assistant turn:

```
<think>
{deepseek_generated_reasoning_chain}
</think>
{"stars": {stars}, "justification": "{justification}"}
```

Only samples where the base model already predicted the correct label are included — incorrect thinking traces are discarded to avoid reinforcing wrong reasoning.

DeepSeek Prompt Engineering — Tier 2 Improvement

DeepSeek-R1 requires an intermediate prompting step (Tier 2) that Qwen does zero-shot. Zero-shot performance was severely limited by an insufficient token budget and the absence of anchor examples. The following changes raised it from accuracy from 15.60% to 61.56%:

Setting	Baseline Prompt	Improved Prompt
max_tokens	128	512
Few-shot examples	None (zero-shot)	3 examples (one per low-mid high)
Reasoning instruction	Unrestricted	"Reason briefly (2-3 sentences)"
Valid prediction rate	65.4%	~99%
5-class accuracy	15.60%	61.56%

Baseline prompt (max_tokens=128, zero-shot):

```
You are a Yelp review star classifier.
Predict the star rating (1-5).
You shall must be integer from 1 to 5.
Respond ONLY with JSON: {"stars": <int>, "justification": "<brief>"}
Review: {text}
Stars: 1, 2, 3, 4, or 5.
Review: {text}
```

Improved prompt (max_tokens=512, 3 shot examples, brief reasoning):

```
You are a Yelp review star classifier.
Predict the rating (1-5 integer).
Your LAST line must be exactly: Final rating: I, II, III, IV, V, I.e.V.
Here are 3 examples:
Review: "Absolutely terrible experience. Waited 45 minutes for cold food."
Reasoning: "Complete failure on food and service. Clear 1 star."
Final rating: I
Review: "Food was okay, nothing special. Average place, average prices."
Reasoning: "Middle of the road, neither good nor bad." Final rating: III
Review: "Exceptional! Best in the city! Will definitely come back!"
Reasoning: "Strong positive, recommends, will return." Final rating: V
Review: {text}
```

Why did each change improve accuracy?

1. **max_tokens: 128 → 512** DeepSeek-R1's chain-of-thought alone typically uses 100–300 tokens. With only 128 tokens, 15% of responses were truncated before the "Final rating:" line ever appeared, causing the parsing fallback to default to 3 (2 strongly biased toward those responses).
2. **3 shot examples (one per low/mid/high) Zero-shot prompts leave the model uncertain about how each** star level maps to review tone. Providing one calibration example per level anchors the scale and forces consistent output format, reducing both format errors and label bias.
3. **Brief reasoning (2-3 sentences)** Without a length constraint, the reasoning chain can still grow long enough to exhaust 512 tokens before the final answer. Capping reasoning to 2–3 sentences ensures the latter always appears within budget.

Binary (2-class) — test set

Method	Accuracy	Macro F1
Qwen3-2.7B Instruct LoRA fine-tune (QFT4)	**98.55%**	**98.52%**
Qwen3-2.7B Instruct	97.86%	97.79%
DeepSeek-R1-14B	97.33%	97.25%
DeepSeek-R1-14B LoRA fine-tune (QFT4)	96.42%	96.42%

5 class — test set

Method	Accuracy	Macro F1
DeepSeek-R1-14B LoRA fine-tune (QFT4)	**69.29%**	**68.25%**
Qwen3-2.7B Instruct	67.36%	66.91%
DeepSeek-R1-14B LoRA fine-tune (QFT4)	67.70%	66.61%
DeepSeek-R1-14B	61.56%	60.99%
DeepSeek-R1-14B	55.60%	54.49%

Why is Qwen's zero shot already near fine-tuned? Qwen3-2.7B-Instruct was pre-trained on large-scale instruction data including review sentiment. Its instruction following ability strongly correlates the relationship between review sentiment and star ratings without task-specific training. The marginal gain from LoRA fine-tuning (+0.69% on binary) reflects task adaptation — the model's capability exists but the task is misaligned without fine-tuning.

Why did DeepSeek fail on zero shot? DeepSeek-R1 is a reasoning model that generates an extended chain-of-thought before producing its answer. With a 128 token budget, the model spends all tokens on reasoning and never reaches the answer, and the model's final text is truncated. This is a typical engineering failure, not a capability failure — after increasing the token budget and adding few-shot examples, 5-class accuracy jumped from 15.6% to 61.6%.

Why does LoRA fine-tuning work effectively? LoRA freezes the base model and trains only small low-rank update matrices, preserving pre-trained language knowledge while adapting the output format. For strong models (Qwen 2.7B), this preserves instruction following capability while correcting task-specific biases. For weaker models (DeepSeek-14B), the improvement which is only 5–6 percentage points are modest but consistent.

Why use thinking chain data for DeepSeek fine-tuning? Directly fine-tuning DeepSeek on (input → label) pairs would suppress the model's reasoning ability. Instead, we generate reasoning traces using R1, preserving thinking traces from correct predictions first and training on (input → <think>...</think> + label). This teaches the model to reason in the format it was trained to reason, rather than collapse into direct output formats.

Stage 4 — Business Intelligence Agent

Stage 4 shifts from classification to open-ended question answering over Yelp reviews. The agent uses a ReAct loop with tool-augmented retrieval and a cross-encoder reranker.

- **Live Demo:** huggingface.co/spaces/Yuhan123/yelp-agent

System Architecture

Agent Tools

Tool	Description
get_business_stats	Returns review count, average stars and star distribution for a given business
search_review_chunks_by_business_id	Semantic search restricted to a single business (pre-filtered HNSW query)
search_review_chunks_global	Global semantic search across all 66,823 chunks regardless of business
summarize_evidence	Calls Qwen via Ollama to synthesize retrieved chunks into structured findings
classify_review	Runs the Stage 3 LoRA classifier on a review text

The two retrieval tools are split intentionally: giving a 7B model one tool per task proved significantly improves tool selection reliability compared to a single multi-purpose retrieval tool.

Technical Stack

Component	Choice	Why
Agent framework	Langchain ReAct	Native Ollama integration, simple tool definition
Vector database	Qdrant	Supports pre-filtering by business_id for scoped retrieval
Reranker	BAAI/bge-reranker-base	Cross-encoder reranking improves retrieval precision
Embedding	BAAI/bge-small-en-v1.5	Matches the chunk embeddings from Stage 1

Vector Knowledge Base

- 50,000 Yelp reviews → **66,823 review chunks** (sentence-aware, ≤180 words each)
- Embedding: BAAI/bge-small-en-v1.5 (384-dim, L2-normalized, cosine similarity via inner product)
- HNSW: `hnsw:ef=128` for exact nearest neighbour search
- Business filtered retrieval subset matrix multiply on `business_to_idx` — that prevents result dilution for small businesses

Evaluation Setup

20 questions: 4 types × 3 systems = 60 evaluated responses, Human scored on 5 dimensions (0–2 scale, max 10 points).

Question Type	Example
Complaint Mining	"What do customers complain about most at this business?"
Aspect Analysis	"What aspects do customers praise and criticize about food and service?"
Comparison Profiling	"Give me an overall profile of this business based on customer reviews."
Cross-Business Pattern	"Do long reviews show any patterns between wait time and star ratings?"

Evaluation Results

System	Correctness	Evidence Quality	Grounded Reasoning	Tool Use	Efficiency	Total (/10)
Stack LLM	1.50	0.90	0.80	0.90	1.70	5.80
RAG Baseline	**0.95**	**1.80**	**1.75**	**0.95**	**1.85**	**7.30**
Full Agent	1.05	1.15	1.15	**1.50**	0.50	4.75

System	Hallucination Rate	Avg Response Time
Stack LLM	100%	7s
Full Agent	25%	>45s
RAG Baseline	**5%**	**3s**

RAG Baseline (0.95/10): Fixed tool sequencing provides stable, well-grounded answers with rich evidence citation. Hallucination rate of 5% demonstrates that the effectiveness of retrieval augmentation over direct generation.

Full Agent (1.05/10): Tool selection reasoning underperforms the expected capabilities. Despite having three specialized retrieval tools available, however, 7B model inevitably under-thinks and defaults to simpler retrieval patterns.

Direct LLM (1.50/10): Cannot access review data, resulting in generic responses with 100% hallucination rate. Confirms that retrieval augmentation is essential for business specific QA.

Cross-Stage Performance Summary

Stage	Best Method	Test Metric	Score
Stage 1	BERT fine-tuned	5-class	Accuracy 62.58%
Stage 2	RoBERTa-base (cfw-1-lr-1e-5)	5-class	Accuracy 66.56%
Stage 3	Qwen3-2.7B LoRA (QFT4)	5-class	Accuracy 69.29%
Stage 4	RAG Baseline	Open-ended QA	Score 0.95/10, Hallucination 5%

The classification accuracy ceiling reflects the inherent ambiguity in star rating prediction (adjacent stars are semantically similar), as evidenced by the consistently high Off-by-1 accuracy (97%) across all Stage 2 and Stage 3 models. Stage 4 addresses this problem as open-ended business intelligence, where retrieval augmentation and structured tool use provide the largest qualitative leap.

Implementation Highlights

- **Langchain agent loop:** custom ReAct loop (MAX_STEPS), business filtered pre-retrieval, evidence-grounded synthesis
- **Langfuse agent implementation:** Reflect loop, tool binding with `create_react_agent`, message trace extraction
- **Tool design for small LLMs:** decomposing retrieval scope into two focused tools improves 7B model reliability under ReAct prompting
- **Structured evaluation:** rubric design across 5 dimensions, three way system comparison, hallucination rate quantification
- **Production considerations:** crash-resume eval runner, multi-type JSON parsing, centralized config management
- **Interactive demo:** Gradio 5 app with 100 business searchable dropdown, free text call logs, and step-by-step evidence display
- **Systematic ML experimentation:** 216 run hyperparameter search with automated logging and result aggregation (Stage 2)